

PERSONAL DIGITAL VAULT

Full Technical Stack Document

Rebuilt Edition - NuGet Packages & Libraries Clarified

Item	Project Detail
Project Type	Secure Full-Stack Web Application
Runtime / Framework	.NET 8 / ASP.NET Core 8 Web API
Frontend	HTML5 + CSS3 + Vanilla JavaScript
Database	Microsoft SQL Server LocalDB / SQL Server Express
ORM	Entity Framework Core 8
API Style	Internal REST API + Swagger/OpenAPI
Document Version	1.1 (Package details clarified)
Prepared	11 August 2026

Submission-ready technical reference for project cloning, document submission and viva.

Document Purpose and Source Basis

This document describes the implemented Personal Digital Vault technical stack. It consolidates the .NET platform, frontend and backend technologies, SQL Server database, programming languages, REST API services, NuGet packages, security technologies, development tools, solution architecture and key project files in one clear submission document.

Important scope note: The inspected build does not use React, Angular, Vue, Blazor, Bootstrap, jQuery, Node.js/npm, a cloud database, a cloud file-storage provider, or an external third-party business API.

Required Technical Stack - Summary

Requirement	Technology Used	Role in Project
.NET Version	.NET 8 / TargetFramework net8.0	Application runtime and ASP.NET Core platform
Frontend	HTML5, CSS3, Vanilla JavaScript	Responsive interface served from wwwroot
Backend	ASP.NET Core 8 Web API	REST endpoints, authentication, authorization, business logic, static hosting
Database	Microsoft SQL Server LocalDB / SQL Server Express	Relational storage via EF Core 8 and migrations
Languages	C#, JavaScript, HTML, CSS, SQL/T-SQL, PowerShell, Batch	Backend, frontend, database scripts and setup automation
APIs / Services	Internal REST API, JWT, encryption, hashing, file storage, Swagger	Communication and security services
Libraries / Tools	EF Core 8, JwtBearer 8.0.0, Swashbuckle 6.5.0, dotnet-ef 8.0.0, Visual Studio	ORM, authentication, API docs, migrations and development

1. .NET Version Used in the Project

1.1 Exact target framework

The backend targets .NET 8. The project file uses TargetFramework net8.0 and the Microsoft.NET.Sdk.Web SDK, so the application is compiled and executed as an ASP.NET Core web application on .NET 8.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

1.2 How .NET 8 is used

- ASP.NET Core hosting creates the WebApplication, dependency-injection container, middleware pipeline, controller routing, HTTPS redirection, CORS, authentication, authorization and static-file hosting.
- Built-in dependency injection registers VaultDbContext and custom services such as JwtTokenService, EncryptionService, HashService and FileStorageService.
- The configuration system uses appsettings.json and appsettings.Development.json for database, JWT, encryption, storage and development settings.
- System.Security.Cryptography provides AES-256-GCM encryption and SHA-256 hashing.
- Async/await and Task are used for database and file operations.

Setting	Value	Meaning
SDK	Microsoft.NET.Sdk.Web	ASP.NET Core web application SDK
Target Framework	net8.0	Compiles for .NET 8

Nullable	enable	Nullable reference type analysis
Implicit Usings	enable	Adds common namespaces automatically
EF CLI Tool	dotnet-ef 8.0.0	Creates/applies EF Core migrations

2. Frontend Technology / Framework

2.1 Technologies used

The frontend is built directly with HTML5, CSS3 and Vanilla JavaScript. It is stored under wwwroot and served by the same ASP.NET Core application. No separate frontend framework or npm build process is required.

Frontend Technology	Where Used	Purpose
HTML5	wwwroot/index.html	Page structure, forms, navigation, dialogs, dashboard and vault screens
CSS3	wwwroot/css/styles.css	Responsive layout, cards, forms, sidebar, modals and UI states
Vanilla JavaScript	wwwroot/js/app.js	UI state, validation, CRUD actions, upload, admin actions and user interactions
Fetch API wrapper	wwwroot/js/api.js	Calls REST endpoints, adds Bearer JWT, handles JSON, errors and downloads
Configuration object	wwwroot/js/config.js	API base URL, file limits, allowed extensions and session keys
SVG assets	wwwroot/assets/logo.svg and inline icons	Scalable branding and interface icons

2.2 Browser technologies

- DOM APIs and event listeners for forms, navigation, tabs, modals and dynamic rendering.
- Fetch API for GET, POST, PUT, PATCH and DELETE requests.
- FormData for multipart document uploads.
- localStorage and sessionStorage for JWT session persistence.
- Blob and URL.createObjectURL for document preview/download.
- Responsive viewport metadata and CSS rules for desktop and mobile-friendly layouts.

3. Backend Technology / Framework

3.1 ASP.NET Core 8 Web API

The backend is an ASP.NET Core 8 Web API. Controller-based REST endpoints are exposed under /api. The backend handles authentication, authorization, business rules, SQL Server access through EF Core, encryption, integrity checking, encrypted file storage and static frontend hosting.

Component	Implementation	Responsibility
Controllers	Auth, Profile, Folders, Categories, Documents, Credentials, Admin	Receive HTTP requests and return API responses
Middleware	ExceptionHandlerMiddleware	Centralized error handling
Authentication	JWT Bearer	Validates signed access tokens
Authorization	[Authorize] + roles	Protects user and administrator endpoints
ORM / Data Access	Entity Framework Core 8	Maps C# entities to SQL Server and executes queries/updates
Dependency Injection	ASP.NET Core DI	Provides DbContext and services to controllers

API Documentation	Swagger / OpenAPI	Interactive API discovery and testing in Development
Static Hosting	UseDefaultFiles + UseStaticFiles	Serves HTML/CSS/JS from wwwroot

3.2 Controller modules

Controller	Main Responsibility
AuthController	Registration, login, logout and JWT session creation
ProfileController	Read/update current authenticated user profile
FoldersController	User-owned folder CRUD
CategoriesController	Credential category CRUD
DocumentsController	Search/list, upload, metadata, preview, download, integrity, rename and delete
CredentialsController	Masked list, secure reveal, create, update and delete encrypted credentials
AdminController	Dashboard, user activation/deactivation and application settings

4. Database Used

4.1 Database platform

The project uses Microsoft SQL Server through SQL Server LocalDB during development. The configured database name is PersonalDigitalVaultMssqlDb and the connection uses Windows trusted authentication. Entity Framework Core 8 is the data-access layer.

```
Server=(localdb)\MSSQLLocalDB;
Database=PersonalDigitalVaultMssqlDb;
Trusted_Connection=True;
TrustServerCertificate=True;
MultipleActiveResultSets=true
```

4.2 Entity Framework Core 8

VaultDbContext is the central EF Core DbContext. It defines application DbSet properties, relationships, indexes, required fields, unique constraints and delete behaviors. EF Core migrations version the SQL Server schema, including the initial MSSQL schema.

4.3 Main database tables and relationships

Table / Entity	Important Data	Relationship
Users	Name, email, password hash, phone, role, active status, timestamps	Parent for user-owned vault data
Folders	Folder name, owner, timestamps	User 1-to-many Folders
Categories	Credential category name, owner, timestamps	User 1-to-many Categories
Documents	Names, path, content type, size, SHA-256, owner, optional folder	User 1-to-many Documents; optional Folder 1-to-many Documents
CredentialRecords	Title/category + encrypted username, secret, website and notes	User 1-to-many CredentialRecords
UploadLogs	Successful upload history	Indexed by UserId
ApplicationSettings	Administrator-managed key/value settings	Key is primary identifier

VaultUser 1 ----< VaultFolder
 VaultUser 1 ----< VaultCategory
 VaultUser 1 ----< StoredDocument
 VaultUser 1 ----< CredentialRecord
 VaultFolder 1 ----< StoredDocument (optional FolderId)

Storage design: document binary content is not stored directly in SQL Server. SQL Server stores metadata and the encrypted document bytes are saved under ProtectedStorage in user-specific folders.

5. Programming Languages Used

Language	Where It Appears	What It Does
C#	Controllers, Data, Models, Services, Middleware, DTOs, Program.cs	Backend API, business logic, auth, database access, encryption, hashing and file operations
JavaScript	wwwroot/js/*.js	Frontend logic, API calls, CRUD, uploads, preview/download and session management
HTML5	wwwroot/index.html	Frontend semantic structure and forms/components
CSS3	wwwroot/css/styles.css	Responsive visual design and UI states
SQL / T-SQL	Database scripts and generated migration SQL	Database inspection and SQL Server persistence
PowerShell	Reset-Development-Database.ps1	Database reset/setup automation
Windows Batch	*.bat scripts	Migration, startup, HTTPS trust and database convenience commands

6. APIs / Services Used

6.1 Internal REST API

The browser frontend communicates with the backend through the project's own REST API. The base path is /api. Normal requests use JSON, document uploads use multipart/form-data, and protected requests include Authorization: Bearer <JWT>.

API Area	Endpoint Summary	Purpose
Authentication	POST /api/auth/register; POST /api/auth/login; POST /api/auth/logout	Account creation and session authentication
Profile	GET /api/profile; PUT /api/profile	Read/update current user profile
Folders	GET/POST /api/folders; GET/PUT/DELETE /api/folders/{id}	Folder CRUD
Categories	GET/POST /api/categories; GET/PUT/DELETE /api/categories/{id}	Credential category CRUD
Documents	GET/POST /api/documents; GET /{id}; preview; download; integrity; rename; delete	Secure document management
Credentials	GET/POST /api/credentials; GET /{id}; reveal; PUT; DELETE	Encrypted credential management
Administrator	dashboard; users; user status; settings	Administrative monitoring and controls

6.2 Application services

Service	Role	Technology / Detail
JwtTokenService	Creates signed JWTs with identity/role claims and expiry	JWT, HMAC SHA-256 signing

EncryptionService	Encrypts/decrypts credential fields and file bytes	AES-256-GCM; 32-byte key; 12-byte nonce; 16-byte tag
HashService	Calculates document integrity fingerprint	SHA-256
FileStorageService	Stores/reads/deletes encrypted files with safe-path validation	ProtectedStorage/{userId}/{guid}.vault
PasswordHasher<VaultUser>	Hashes and verifies account passwords	ASP.NET Core Identity PasswordHasher
VaultDbContext	Database session and entity mapping	EF Core 8 + SQL Server provider
Swagger/OpenAPI	Interactive API documentation and testing	Swashbuckle.AspNetCore 6.5.0

6.3 External APIs

No external third-party business API integration was found in the inspected build. The primary API used by the frontend is the project's own ASP.NET Core REST API. Swagger is documentation/testing tooling, not an external business service.

7. NuGet Packages, Libraries and Tools - Exact Versions

IMPORTANT: The following five NuGet packages are explicitly listed here with their exact package names, versions and purposes. This section is intentionally separated for easy submission and viva reference.

7.1 Installed NuGet packages (exact names and versions)

Exact NuGet Package Name	Version
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.0
Microsoft.EntityFrameworkCore.Design	8.0.0
Microsoft.EntityFrameworkCore.SqlServer	8.0.0
Microsoft.EntityFrameworkCore.Tools	8.0.0
Swashbuckle.AspNetCore	6.5.0

1. Microsoft.AspNetCore.Authentication.JwtBearer - Version 8.0.0

Purpose: JWT Bearer authentication and token validation. It enables ASP.NET Core to read the Authorization: Bearer <token> header and validate protected API requests.

2. Microsoft.EntityFrameworkCore.Design - Version 8.0.0

Purpose: EF Core design-time support. Required by migration/design tooling when creating or inspecting migrations.

3. Microsoft.EntityFrameworkCore.SqlServer - Version 8.0.0

Purpose: Microsoft SQL Server database provider for EF Core. It enables UseSqlServer(...) and SQL Server queries/updates through VaultDbContext.

4. Microsoft.EntityFrameworkCore.Tools - Version 8.0.0

Purpose: Entity Framework migration and database-management tooling used with migration commands and development database operations.

5. Swashbuckle.AspNetCore - Version 6.5.0

Purpose: Generates Swagger/OpenAPI documentation and provides Swagger UI for interactive endpoint viewing/testing in Development.

7.2 Important distinction: package vs built-in framework library

Item	Type	Version / Source	Use
dotnet-ef	Local .NET CLI tool	8.0.0	Creates, applies and manages EF Core migrations from the command line
PasswordHasher<VaultUser>	Built-in ASP.NET Core Identity service	.NET / ASP.NET Core 8	Securely hashes and verifies user passwords; not listed as a separate NuGet package in this build
System.Security.Cryptography	Built-in .NET library	.NET 8	AES-256-GCM encryption and SHA-256 hashing primitives
Fetch API / FormData / Web Storage	Browser APIs	Browser platform	Frontend API calls, uploads and session storage

7.3 How the packages appear in the project

```
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.0" />
```

```
<PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0" />
```

```

<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0" />
<PackageReference Include="Swashbuckle.AspNetCore" Version="6.5.0" />

```

Restore behavior: the package references live in the .csproj file. When a cloned project runs dotnet restore (or Visual Studio restores NuGet packages), the required package binaries are downloaded into the local NuGet cache.

8. Other Framework and Platform Technologies

Technology	Use in Project
ASP.NET Core Dependency Injection	Registers DbContext and custom services with appropriate lifetimes
ASP.NET Core Identity PasswordHasher	Secure password hashing without a custom algorithm
JWT / Bearer Authentication	Stateless login token sent in Authorization header
Role-based Authorization	Administrator-only routes use role checks / [Authorize]
AES-256-GCM	Authenticated encryption of sensitive credential fields and document bytes
SHA-256	Integrity checking for original document content
CORS	Allows configured local-development browser origins
HTTPS	Development HTTPS endpoint with redirection enabled
Swagger / OpenAPI	Interactive endpoint discovery and API testing in Development
SQL Server LocalDB	Local SQL Server development instance
EF Core Migrations	Version-controlled database schema changes
Visual Studio Solution	PersonalDigitalVault.sln for development and debugging

9. Solution Architecture

The project uses a compact layered architecture. The browser loads static frontend files from ASP.NET Core, JavaScript calls /api endpoints, controllers apply authentication/business rules, services provide security and file handling, EF Core communicates with SQL Server, and encrypted document bytes are stored separately under ProtectedStorage.

BROWSER

- | HTML + CSS + JavaScript
- | fetch() + JSON / FormData + Bearer JWT
- v

ASP.NET CORE 8 APPLICATION

- |-- Static File Hosting (wwwroot)
- |-- API Controllers (/api/...)
- |-- Authentication + Authorization
- |-- Exception Middleware + CORS
- |-- Services
 - | |-- JwtTokenService
 - | |-- EncryptionService (AES-256-GCM)
 - | |-- HashService (SHA-256)
 - | |-- FileStorageService
- |
- |-- Entity Framework Core 8 -----> SQL Server LocalDB
- | Users / Folders / Categories / Documents / Credentials / Logs / Settings
- |
- `--- Encrypted file bytes -----> ProtectedStorage/<user>/<file>.vault

10. Security Technology Flow

10.1 Login and authorization

1. User submits email and password from the frontend.
2. AuthController validates the user and password hash.
3. JwtTokenService issues a signed token with user/role claims and expiry.
4. JavaScript stores the session in sessionStorage or localStorage.
5. api.js attaches Authorization: Bearer <token> to protected requests.
6. JwtBearer validates token signature, issuer, audience, lifetime and signing key.
7. Role-based authorization protects administrator routes.

10.2 Document upload and integrity

8. Frontend sends a file using multipart/form-data.
9. DocumentsController validates ownership, extension and file-size constraints.
10. SHA-256 is calculated from the original file bytes.
11. AES-256-GCM encrypts the document before disk storage.
12. FileStorageService stores the encrypted bytes in a user-specific ProtectedStorage path.
13. SQL Server stores metadata, file path, size, hash, owner and optional folder.
14. Preview/download decrypts the file and verifies integrity before returning it to the owner.

11. Project Folder Structure and Key Technical Files

```

PersonalDigitalVault_Demo_Auth_Database/
|-- PersonalDigitalVault.sln
|-- .config/dotnet-tools.json
|-- Database/
|   |-- InspectDemoDatabase.sql
|-- docs/
|   |-- API_ENDPOINTS.md
|   |-- DATABASE_SCHEMA.md
|   |-- IMPLEMENTATION_DECISIONS.md
|-- src/PersonalDigitalVault.Api/
|   |-- PersonalDigitalVault.Api.csproj
|   |-- Program.cs
|   |-- appsettings.json
|   |-- appsettings.Development.json
|   |-- Controllers/
|   |-- DTOs/
|   |-- Data/
|   |-- Extensions/
|   |-- Middleware/
|   |-- Migrations/
|   |-- Models/
|   |-- Services/
|   |-- wwwroot/
|       |-- index.html
|       |-- css/styles.css
|       |-- js/config.js
|       |-- js/api.js
|       |-- js/app.js
|-- tests/PersonalDigitalVault.Api.http
|-- Apply-Migrations.bat
|-- Add-Migration.bat
|-- Reset-Development-Database.ps1
-- Start-PersonalDigitalVault.bat

```

12. Key Configuration Values

Configuration Area	Implemented Value / Behavior
Database	SQL Server LocalDB database PersonalDigitalVaultMssqlDb; EF migrations can auto-apply in Development
JWT	Issuer PersonalDigitalVault.Api; Audience PersonalDigitalVault.Client; default 60-minute expiry; symmetric signing secret
Encryption	Base64 configuration key decoded to exactly 32 bytes for AES-256
Storage Root	ProtectedStorage
Maximum File Size	10,485,760 bytes (10 MB) by default
Allowed Extensions	.pdf, .doc, .docx, .txt, .jpg, .jpeg, .png
Development URL	https://localhost:7187 and http://localhost:5187
Swagger URL	/swagger in Development

13. Technology Responsibility Matrix

Project Feature	Technology Responsible
User registration and login	ASP.NET Core Controllers + PasswordHasher + EF Core + JWT
Secure session	JWT Bearer Authentication + JavaScript sessionStorage/localStorage
Document upload	HTML file input + JavaScript FormData + DocumentsController
Document confidentiality	AES-256-GCM + FileStorageService
Document integrity	SHA-256 HashService
Document metadata	SQL Server + EF Core
Folders and categories	Controller CRUD + EF Core relationships/constraints
Credential confidentiality	AES-256-GCM encrypted database fields
Admin access control	JWT claims + role-based authorization
API testing/documentation	Swagger/OpenAPI via Swashbuckle.AspNetCore
Responsive interface	HTML5 + CSS3 + Vanilla JavaScript
Database schema evolution	EF Core Migrations + dotnet-ef

14. Technologies Not Used in This Build

- No React, Angular, Vue, Blazor or other frontend framework.
- No Bootstrap or jQuery dependency in the inspected frontend source.
- No Node.js / npm build process is required for the frontend.
- No external cloud database is configured; development uses SQL Server LocalDB.
- No cloud file-storage provider is configured; encrypted files are stored under local ProtectedStorage.
- No external third-party business API integration was found.

15. Short Viva Explanation of the Complete Stack

30-second answer

Our Personal Digital Vault is a full-stack web application built on .NET 8. The backend is an ASP.NET Core 8 Web API written in C# and uses Entity Framework Core 8 with Microsoft SQL Server LocalDB. The frontend is HTML5, CSS3 and Vanilla JavaScript served from ASP.NET Core wwwroot. The frontend calls our own REST API using the Fetch API. Authentication uses JWT Bearer

tokens and ASP.NET Core PasswordHasher. Sensitive credentials and document files are protected with AES-256-GCM encryption, while SHA-256 verifies document integrity. Swagger/OpenAPI is used for API documentation and testing, and EF Core migrations manage the database schema.

Package answer for viva: Microsoft.AspNetCore.Authentication.JwtBearer 8.0.0, Microsoft.EntityFrameworkCore.Design 8.0.0, Microsoft.EntityFrameworkCore.SqlServer 8.0.0, Microsoft.EntityFrameworkCore.Tools 8.0.0, and Swashbuckle.AspNetCore 6.5.0. dotnet-ef 8.0.0 is the EF Core CLI tool.

16. Final Technical Stack Statement

The Personal Digital Vault is a self-contained ASP.NET Core full-stack application: .NET 8 and C# power the server; HTML, CSS and JavaScript provide the browser interface; SQL Server LocalDB and Entity Framework Core 8 provide relational persistence; JWT, AES-256-GCM, SHA-256 and role authorization provide core security; and Swagger, EF Core migrations, Visual Studio, CLI tools, batch scripts and PowerShell support development, testing and database setup.

The five primary NuGet packages are stated explicitly in Section 7.1 with exact names, versions and purposes so they can be identified quickly during technical document submission or viva.